

heading_grammar_gate.sh — tracker heading code-bearing guard

Revision: 1 Last modified: 2026-09-01T20:48:54Z

Companion guide (§11.4.18) for scripts/testing/
heading_grammar_gate.sh and its paired test scripts/testing/
heading_grammar_gate_test.sh.

Overview

A §11.4.135 permanent regression guard for one captured defect class:
a `###` item heading in `Issues.md` or `Fixed.md` that carries **no block code**.

`cmd/workable-items/parser.go` reads an item heading with exactly this grammar:

```
^###\s+([A-Z])(\d+)(?:\.\s+|\s+)(\S.*)$
```

— one uppercase letter, at least one digit, then either `.` or `,` then a non-blank title. A `###` line that does not match is **not skipped harmlessly**: the parser keeps walking, so everything under that heading — its `**Status:**`, its `**Type:**`, its whole body — is absorbed into the `raw_body` of the **preceding coded block**. The item then has no row, no TMX-ID, no status of its own, and cannot be reported on, closed, or validated. It is invisible to the SQLite single source of truth (§11.4.93) while still *looking* like a tracked item to a human reader.

The three captured instances (2026-09-01)

One architectural root cause — the tracker grammar never *required* a `###` heading to carry a block code — reached by three unrelated triggers:

#	File	Heading	Trigger	Absorbed into
1	Issues.md	### M24-ESCAPE-001 – ...	a closed record hand-written into the open tracker — a §11.4.19 atomic-migration miss	D2 TMPDIR-HARDCODE-001
2	Fixed.md	### TMX-051 – Per-session color ...	commit 89324dc, the ATM- → TMX- ticket-prefix migration, rewrote a heading whose text <i>happened</i> to begin with a ticket-id literal	D1.
3	Fixed.md	### NEZHA-INSTALL-v1.0.26-001 (closed) – ...	commit 7aefdf2, a §12.10 doc update	D1.

Instance 2 is the sharpest illustration that the leading literal is **not** a binding. TMX-051 is the id of a *different* item (A43, "Copy/paste: terminal owns the mouse by default"); the colour block simply carried the string. Measured with `workable-items` report: 95 rows existed and **none** of the three blocks had one.

Measured absorption, before and after the repair:

```
PRE : D1. block spans 7703 chars; contains 'Per-session
color'=True 'nezha'=True
POST: D1. block spans 2200 chars; contains 'Per-session
color'=False 'nezha'=False
```

```
md-to-db BEFORE: fixed_parsed=83 inserted=0 allocated=0
md-to-db AFTER : fixed_parsed=86 inserted=3 allocated=3 →
TMX-096/097/098
```

Why nothing existing caught it

Every other check in this repo reasons about blocks the parser **already sees**. To validate, to the `md-to-db` → `db-to-md` round trip, to `scripts/tests/51_workable_items_db_integrity.sh`, and to the sibling `tracker_structural_integrity.sh`, a heading the parser skips is *not a heading at all* — so a corpus with three invisible items and a corpus

with none return the identical quiet zero. That is the §11.4.201(6) **false null**. This gate is the only one that asserts on the raw document grammar, **before** the parser's own filter has a chance to hide the defect.

Prerequisites

- awk and grep (POSIX). Nothing else — no git, no network, no build, no database.
 - bash to invoke it. The script itself is `sh -n` clean as well (§11.4.67): no arrays, no `[[ ]]`, no here-strings, no `local`.
 - A writable `${TMPDIR:-/tmp}`. The scratch directory is honoured from `TMPDIR` and never hardcoded to `/tmp` — the defect Fixed.md D2 (`TMPDIR-HARDCODE-001` / `TMX-092`) records why.
-

Usage

```
# default: Issues.md + Fixed.md at the repo root
bash scripts/testing/heading_grammar_gate.sh

# explicit files (repeatable; absolute or repo-root-relative)
bash scripts/testing/heading_grammar_gate.sh --file Issues.md --
      file Fixed.md

# suppress the per-file inventory line
bash scripts/testing/heading_grammar_gate.sh --quiet

# the header itself
bash scripts/testing/heading_grammar_gate.sh --help
```

Exit codes: **0** = PASS or SKIP-with-reason, **1** = FAIL (at least one code-less heading). A SKIP always prints its reason and is never silently counted as a pass (§11.4.3).

Output

```
[evidence heading-grammar] control needle: 3 synthetic headings
pushed through the SAME scanner → seen=3 coded=2 code-less=1
(expected seen=3 coded=2 code-less=1)
[evidence heading-grammar] Issues.md: scanner SEES headings=10
coded=10 code-less=0
PASS heading-grammar: G1 Issues.md — all 10 `###` heading(s)
carry a parser-readable block code
[evidence heading-grammar] Fixed.md: scanner SEES headings=86
```

coded=86 code-less=0
PASS heading-grammar: G1 Fixed.md — all 86 `###` heading(s) carry
a parser-readable block code
— summary heading-grammar: PASS=2 FAIL=0 SKIP=0 —

On a refusal, every offender is printed with **resolved evidence** (§11.4.201(5)) — the exact path, line number, and heading text — so the finding is actionable in one step:

```
[evidence heading-grammar] OFFENDER /.../Fixed.md:2835: ### TMX-051
- Per-session color via `name:color[:ignored]`
FAIL heading-grammar: G1 Fixed.md — 2 of 85 `###` heading(s)
carry NO block code, ...
```

Fixing a finding

Give the heading a **free** <LETTER><DIGITS> code. Both forms are valid and the gate accepts either:

```
### D2. Per-session color via `name:color[:ignored]` -
      `IMPLEMENTED`      ← period form
### B54 M24-ESCAPE-001 - meta-test M24 ... -
      `FIXED`            ← space form
```

Then add a ****TMX-ID:**** line so the binding is explicit rather than allocation-order dependent, and — if the block carries a terminal status — make sure it lives in `Fixed.md`, not `Issues.md` (§11.4.19). Free codes are found by listing the existing ones per file:

```
grep -oE '^### [A-Z][0-9]+' Fixed.md | sort -u
```

Block codes are a **per-file** namespace: `Issues.md` and `Fixed.md` both legitimately carry an H1 and an I1.

Edge cases

Deliberately **not** flagged — a false-positive refusal is a FAIL-bluff exactly as a false-negative pass is a PASS-bluff (§11.4.201(1)):

Input	Verdict	Why
### A7. Title	coded	period form, the parser's \. \s+ branch
### G5 Title	coded	space form, the parser's \s+ branch
### B123 Title	coded	multi-digit ordinals are fine
## D. Section	ignored	section header; the item grammar does not apply

Input	Verdict	Why
#### Sub-heading	ignored	not an item heading (^### requires a non-# next char)
###foo	ignored	not a Markdown ATX heading at all — anyHeadingRE needs #{1,6}\s
### X (indented)	ignored	the parser anchors at column 0
### X inside ``` or ~~~	ignored	sample text inside a fenced block, not a heading

Correctly flagged as **code-less**, because the parser cannot read them either:

Input	Why
### NEZHA-INSTALL-001 – ...	N is followed by E, not a digit
### AB1 Title	([A-Z]) is <i>one</i> letter; B is not a digit
### A7.Title	the separator needs whitespace after the .
### A7	no title after the code

The gate mirrors headingRE **exactly** rather than inventing a stricter rule of its own — the property under test is "can parser.go read this heading", so the check probes the real grammar rather than a proxy (§11.4.201(11)).

Internal behaviour

One scanner, one code path

A single awk program (_scan) does all classification and is used for **both** the real trackers and the control needle. There is no second, divergent implementation that could drift.

It walks each file once, tracking fenced code blocks (``` and ~~~, any fence length, indented or not — a fence of one character closes only on the same character), skips #####+ and non-column-0 lines, and classifies

each surviving `###` line with `_hg_coded`. It emits HEADINGS/CODED/CODELESS counts plus one OFFENDER `<line>\t<text>` row per code-less heading.

The control needle (§11.4.201(7)(b))

A zero-finding result is refused unless the instrument is proven to see — **in both polarities, through the same code path** — before any file is judged:

- **Seeing** — the scanner must extract more than zero `###` headings from the file. Zero extracted means BLIND, and the gate SKIPS with that reason rather than reporting a confident clean zero.
- **Polarity** — three synthetic needle lines (two known-coded, one known-code-less) are pushed through the identical `_scan`. A classifier stuck at *"everything is coded"* would return a clean zero on a genuinely broken file; the code-less needle is what refutes that. If the needle lands 3/2/1 the instrument discriminates; anything else is BLIND → SKIP.

The needle counts are **printed on every run**, so the null is auditable rather than merely asserted. T7 in the paired test breaks the needle deliberately and proves the gate then refuses to emit a PASS at all.

The load-bearing assertion

`_hg_coded`'s regex is the whole verdict. The paired test weakens it (M1) and neuters the code-less counter (M2) in **copies** under `${TMPDIR}` — the tracked gate is never edited, so no mutation residue can be staged (§11.4.84) — and asserts that each weakened copy then **misses** the golden-bad fixture.

Validation evidence

§11.4.115(F) — a guard never observed failing on the genuinely-broken artifact is unvalidated instrumentation. This one was:

```
$ git show HEAD:Issues.md > /tmp/pre_Issues.md
$ git show HEAD:Fixed.md > /tmp/pre_Fixed.md
$ bash scripts/testing/heading_grammar_gate.sh --file /tmp/
pre_Issues.md --file /tmp/pre_Fixed.md
FAIL heading-grammar: G1 pre_Issues.md — 1 of 10 ... code-less
```

FAIL heading-grammar: G1 pre_Fixed.md - 2 of 85 ... code-less
— summary heading-grammar: PASS=0 FAIL=2 SKIP=0 — (rc=1)

T5 of the paired test performs exactly this, automatically, so the observation is re-run on every invocation rather than being a one-off claim.

— exactly the three known offenders, and no false positives. After the repair the same gate reports PASS=2 FAIL=0 SKIP=0 (rc=0).

Paired test: 9 PASS / 0 FAIL, identical across three consecutive runs (§11.4.50).

Related scripts

Path	Relationship
scripts/testing/ heading_grammar_gate_test.sh	the §1.1 paired test for this gate
scripts/testing/ tracker_structural_integrity.sh	sibling guard. That one asserts a tracker did not lose structure across revisions; this one asserts every heading is structure the parser can read. Neither subsumes the other.
cmd/workable-items/parser.go	headingRE — the grammar this gate mirrors
cmd/workable-items/ sync_md_to_db.go	the verbatim absorption that makes a code-less heading invisible
scripts/tests/ 51_workable_items_db_integrity.sh	the self-referential round trip that cannot see this defect class

Honest boundary (§11.4.6)

This gate proves every ### heading is **readable by the parser**. It does **not** prove the heading's code is the *right* one (a duplicate code is validate's job, via the §11.4.54 duplicate/gap checks), that the block's ****Status:**** matches its file (§11.4.19/§11.4.33), or that the item's content is correct. It is one layer, and it is the earliest one.

Sources verified 2026-09-01

- cmd/workable-items/parser.go — headingRE, anyHeadingRE (read in-tree at commit 2c8187a).
- cmd/workable-items/validate.go — the §11.4.54 id sequence/duplicate rules that this gate deliberately does **not** duplicate.
- Measured with workable-items report / sync md-to-db against a **copy** of docs/workable_items.db; the tracked database was not modified.